



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Grado en Matemáticas Computacionales

**Simulación de precios de activos financieros
mediante el método de Monte Carlo:
aplicación y comparación de modelos
estocásticos**

Trabajo de Fin de Grado presentado por:	Inocencio Bielo Bomala
Tipo de trabajo:	Investigación aplicada
Director/a:	Jesús Alejandro Aponte González
Fecha:	08/09/2026

Resumen

Este trabajo estudia la simulación de precios de activos financieros mediante procesos estocásticos y métodos de Monte Carlo. El modelo principal es el movimiento browniano geométrico o *Geometric Brownian Motion* (GBM), cuya solución analítica permite representar el precio de un activo mediante una distribución lognormal. Se desarrollan los fundamentos matemáticos del movimiento browniano, las ecuaciones diferenciales estocásticas, la integral y la fórmula de Itô, la distribución de los rendimientos logarítmicos y la convergencia del método de Monte Carlo.

Posteriormente se implementa el modelo en Python y se valida mediante simulaciones, pruebas estadísticas y comparación entre resultados empíricos y teóricos. La calibración se realiza utilizando una serie histórica del índice S&P 500. Además, se analiza la sensibilidad del modelo respecto a los parámetros de deriva y volatilidad, prestando especial atención a la varianza del precio terminal.

El trabajo incluye además dos comparaciones con objetivos diferentes. La primera, planteada desde la entrega inicial, utiliza la fórmula de Black–Scholes como referencia analítica para contrastar el resultado obtenido mediante simulación de Monte Carlo en la valoración de una opción europea. La segunda comparación se realiza con un modelo GARCH(1,1), incorporado posteriormente para estudiar una limitación concreta del GBM: la hipótesis de volatilidad constante. De esta forma, Black–Scholes permite contrastar la precisión numérica del método de Monte Carlo, mientras que GARCH(1,1) permite analizar qué ocurre cuando la volatilidad se considera variable en el tiempo.

Palabras clave: movimiento browniano geométrico, GBM, Monte Carlo, cálculo de Itô, GARCH(1,1), volatilidad, simulación financiera.

Abstract

This work studies the simulation of financial asset prices using stochastic processes and Monte Carlo methods. The main model is the Geometric Brownian Motion (GBM), whose analytical solution describes asset prices through a lognormal distribution. The mathematical foundations of Brownian motion, stochastic differential equations, the Itô integral and formula, logarithmic returns, and Monte Carlo convergence are developed.

The model is subsequently implemented in Python and validated through simulations, statistical tests, and comparisons between empirical and theoretical results. Calibration is performed using historical data from the S&P 500 index. A sensitivity analysis of drift and volatility parameters is also carried out, with particular attention to the variance of terminal prices.

Finally, a GARCH(1,1) model is incorporated as an alternative to GBM in order to address one of its main limitations: the assumption of constant volatility. Both models are compared using the same historical data and an out-of-sample validation procedure.

Índice

Resumen	1
Abstract	2
1. Introducción	5
1.1. Objetivos	5
2. Fundamentos teóricos	6
2.1. Procesos estocásticos y movimiento browniano	6
2.2. Ecuaciones diferenciales estocásticas	7
2.3. La integral de Itô	7
2.4. Fórmula de Itô	7
2.5. Solución analítica del GBM	8
2.6. Esperanza y varianza	8
2.7. Rendimientos logarítmicos	8
2.8. Método de Monte Carlo	9
3. Primera comparación: Monte Carlo y Black–Scholes	9
3.1. Relación entre GBM y Black–Scholes	10
3.2. Valoración mediante Monte Carlo	11
3.3. Error de la estimación	11
3.4. Diferencia respecto a la comparación con GARCH	12
4. Datos reales y preprocesamiento	13
4.1. Calidad y preprocesamiento de los datos	14
4.2. Índice y variables utilizadas	14
4.3. Período temporal y número de observaciones	15
4.4. Análisis de normalidad de los rendimientos	15
4.5. Días no bursátiles y datos faltantes	16
4.6. División entrenamiento-validación	16
5. Metodología e implementación del GBM	17
5.1. Calibración de μ y σ	17
5.2. Discretización temporal	17
5.3. Estadísticos y validación	17
6. Validación del GBM con datos simulados	18

7. Análisis de sensibilidad del GBM	20
7.1. Sensibilidad respecto a μ	20
7.2. Sensibilidad respecto a σ	21
7.3. Metodología del análisis	22
8. Modelo GARCH(1,1)	23
8.1. Motivación	23
8.2. Definición	23
8.3. Calibración	24
8.4. Simulación	25
9. Comparación entre GBM y GARCH(1,1)	25
9.1. Diseño experimental	25
9.2. Comparación conceptual	26
9.3. Comparación de volatilidad	26
9.4. Comparación fuera de muestra	27
10.Discusión y limitaciones	28
11.Conclusiones	28
A. Código fuente	29
A.1. Implementación del movimiento browniano geométrico	29
A.2. Análisis y validación del GBM	32
A.3. Implementación del modelo GARCH(1,1)	48
A.4. Comparación entre modelos	55

1. Introducción

Los mercados financieros están caracterizados por la incertidumbre. El precio futuro de un activo depende de múltiples factores y no puede determinarse de forma exacta. Sin embargo, mediante modelos matemáticos y probabilísticos es posible representar de forma aproximada su evolución y estudiar sus propiedades estadísticas.

El presente trabajo se centra en la simulación de precios financieros mediante el movimiento browniano geométrico y métodos de Monte Carlo. El GBM constituye uno de los modelos estocásticos más conocidos en finanzas cuantitativas y sirve como punto de partida para numerosos modelos posteriores.

El trabajo combina una parte teórica y una parte experimental. En primer lugar se desarrollan los fundamentos matemáticos necesarios para comprender el modelo. Posteriormente se implementa un simulador en Python, se valida frente a las propiedades teóricas del GBM y se calibra utilizando datos históricos reales.

El desarrollo del trabajo ha seguido dos líneas de comparación. Por un lado, se utiliza Black-Scholes como referencia analítica para comprobar la precisión del procedimiento de Monte Carlo en un caso en el que existe una solución cerrada. Por otro lado, al analizar datos financieros reales, se introduce un modelo GARCH(1,1) para estudiar la evolución de una volatilidad que no tiene por qué ser constante. Esta segunda comparación surge directamente de una de las principales limitaciones observadas en el GBM.

1.1. Objetivos

El objetivo general es estudiar, implementar y comparar modelos estocásticos para la simulación de precios y rendimientos financieros.

Los objetivos específicos son:

1. Desarrollar los fundamentos matemáticos del movimiento browniano y del movimiento browniano geométrico.
2. Explicar la interpretación de las ecuaciones diferenciales estocásticas y las principales herramientas del cálculo de Itô.
3. Deducir la solución analítica del GBM.
4. Implementar un simulador mediante métodos de Monte Carlo.
5. Validar los resultados simulados mediante estadísticas y pruebas de ajuste.
6. Calibrar los parámetros del modelo utilizando datos reales.

7. Analizar la sensibilidad de la media y la varianza respecto a μ y σ .
8. Contrastar una valoración obtenida mediante Monte Carlo con la fórmula analítica de Black–Scholes.
9. Implementar y calibrar un modelo GARCH(1,1).
10. Comparar el comportamiento del GBM y del GARCH(1,1) utilizando un conjunto común de datos y una validación fuera de muestra.

2. Fundamentos teóricos

El movimiento browniano constituye uno de los procesos estocásticos fundamentales utilizados en la modelización financiera [2].

2.1. Procesos estocásticos y movimiento browniano

Un proceso estocástico es una familia de variables aleatorias $(X_t)_{t \geq 0}$ indexada por el tiempo. En este trabajo, el proceso fundamental será el movimiento browniano estándar o proceso de Wiener $(W_t)_{t \geq 0}$.

Un proceso es un movimiento browniano estándar si cumple:

1. $W_0 = 0$.
2. Tiene incrementos independientes.
3. Para $0 \leq s < t$,

$$W_t - W_s \sim N(0, t - s). \quad (1)$$

4. Sus trayectorias son continuas con probabilidad uno.

En particular,

$$W_t \sim N(0, t), \quad (2)$$

por lo que

$$\mathbb{E}[W_t] = 0, \quad \text{Var}(W_t) = t. \quad (3)$$

Para un intervalo Δt ,

$$\Delta W_t = W_{t+\Delta t} - W_t = \sqrt{\Delta t} Z, \quad (4)$$

donde $Z \sim N(0, 1)$. Esta expresión permite generar computacionalmente los incrementos del proceso browniano.

2.2. Ecuaciones diferenciales estocásticas

Una ecuación diferencial estocástica puede expresarse de forma general como

$$dX_t = a(X_t, t) dt + b(X_t, t) dW_t. \quad (5)$$

El término $a(X_t, t) dt$ representa la componente determinista o deriva, mientras que $b(X_t, t) dW_t$ representa la componente aleatoria.

El movimiento browniano geométrico viene definido por

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad (6)$$

donde S_t es el precio del activo, μ representa la deriva y σ la volatilidad.

2.3. La integral de Itô

La integral de Itô de un proceso H_t respecto a un movimiento browniano se escribe como

$$\int_0^T H_t dW_t. \quad (7)$$

Se construye como límite de sumas en las que el integrando se evalúa en el extremo izquierdo de cada intervalo. Esta característica garantiza que el integrando depende únicamente de la información disponible antes del incremento browniano.

Una propiedad fundamental es la isometría de Itô:

$$\mathbb{E} \left[\left(\int_0^T H_t dW_t \right)^2 \right] = \mathbb{E} \left[\int_0^T H_t^2 dt \right]. \quad (8)$$

Además,

$$(dW_t)^2 = dt, \quad dt dW_t = 0, \quad (dt)^2 = 0. \quad (9)$$

2.4. Fórmula de Itô

Si

$$dX_t = a dt + b dW_t, \quad (10)$$

y $f(x, t)$ es suficientemente regular, la fórmula de Itô establece que

$$df = \left(\frac{\partial f}{\partial t} + a \frac{\partial f}{\partial x} + \frac{1}{2} b^2 \frac{\partial^2 f}{\partial x^2} \right) dt + b \frac{\partial f}{\partial x} dW_t. \quad (11)$$

Para $f(S_t) = \log(S_t)$,

$$f'(S_t) = \frac{1}{S_t}, \quad f''(S_t) = -\frac{1}{S_t^2}. \quad (12)$$

Aplicando la fórmula de Itô al GBM se obtiene

$$d \log(S_t) = \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma dW_t. \quad (13)$$

El término $-\frac{1}{2}\sigma^2$ constituye la corrección de Itô.

2.5. Solución analítica del GBM

Integrando (13) entre 0 y T ,

$$\log(S_T) - \log(S_0) = \left(\mu - \frac{1}{2} \sigma^2 \right) T + \sigma W_T. \quad (14)$$

Por tanto,

$$S_T = S_0 \exp \left[\left(\mu - \frac{1}{2} \sigma^2 \right) T + \sigma W_T \right]. \quad (15)$$

Como $W_T = \sqrt{T}Z$, con $Z \sim N(0, 1)$,

$$S_T = S_0 \exp \left[\left(\mu - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T}Z \right]. \quad (16)$$

Esta expresión permite simular directamente el precio terminal.

2.6. Esperanza y varianza

A partir de la solución analítica se obtiene

$$\mathbb{E}[S_T] = S_0 e^{\mu T}, \quad (17)$$

y

$$\text{Var}(S_T) = S_0^2 e^{2\mu T} \left(e^{\sigma^2 T} - 1 \right). \quad (18)$$

La ecuación (18) será especialmente relevante en el análisis de sensibilidad.

2.7. Rendimientos logarítmicos

Definiendo

$$r_t = \log \left(\frac{S_{t+\Delta t}}{S_t} \right), \quad (19)$$

se tiene

$$r_t \sim N \left(\left(\mu - \frac{1}{2} \sigma^2 \right) \Delta t, \sigma^2 \Delta t \right). \quad (20)$$

Por tanto,

$$\mathbb{E}[r_t] = \left(\mu - \frac{1}{2} \sigma^2 \right) \Delta t, \quad (21)$$

y

$$\text{Var}(r_t) = \sigma^2 \Delta t. \quad (22)$$

El precio es, en consecuencia, una variable lognormal.

2.8. Método de Monte Carlo

Si se desea calcular

$$I = \mathbb{E}[f(X)], \quad (23)$$

y se generan N muestras independientes $X_1, \dots, X_N$, el estimador es

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^N f(X_i). \quad (24)$$

La Ley de los Grandes Números garantiza que

$$\hat{I}_N \longrightarrow I \quad (25)$$

cuando $N \rightarrow \infty$.

El Teorema Central del Límite implica que

$$\text{Var}(\hat{I}_N) = \frac{\text{Var}(f(X))}{N}, \quad (26)$$

por lo que el error típico es de orden

$$O(N^{-1/2}). \quad (27)$$

3. Primera comparación: Monte Carlo y Black–Scholes

El método de Monte Carlo permite aproximar cantidades esperadas mediante simulación aleatoria y constituye una herramienta fundamental en ingeniería financiera [3].

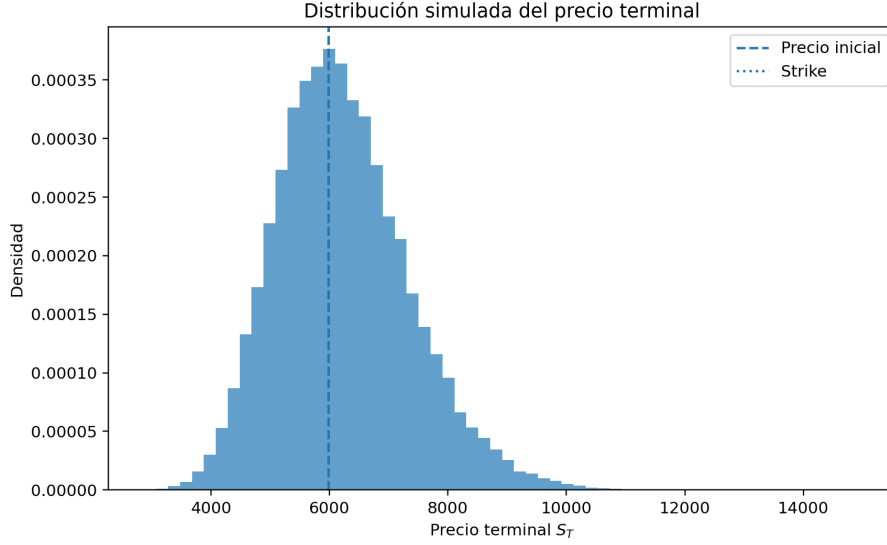


Figura 1: Distribución simulada del precio terminal utilizada en la valoración mediante Monte Carlo.

La primera comparación incluida en el planteamiento original del trabajo no se realiza entre dos modelos de evolución del precio equivalentes. Su objetivo es distinto: comprobar que una estimación obtenida mediante simulación de Monte Carlo es coherente con un valor analítico conocido. En la primera entrega se planteó utilizar el modelo de Black–Scholes como referencia para valorar una opción europea cuyo activo subyacente sigue un movimiento browniano geométrico.

3.1. Relación entre GBM y Black–Scholes

Bajo las hipótesis habituales del modelo de Black–Scholes, el precio del activo subyacente sigue un GBM. Por tanto, el GBM describe la dinámica del activo, mientras que Black–Scholes proporciona una fórmula cerrada para el valor de determinados derivados construidos sobre dicho activo. La comparación realizada en esta parte del trabajo consiste, por tanto, en valorar el mismo contrato de dos formas: mediante la fórmula analítica y mediante la simulación de las trayectorias del subyacente.

Para una opción europea de compra, con precio inicial S_0 , precio de ejercicio K , tipo de interés libre de riesgo r , volatilidad σ y vencimiento T , el valor analítico viene dado por

$$C_{BS} = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2), \quad (28)$$

donde

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}, \quad (29)$$

y Φ representa la función de distribución de una normal estándar.

3.2. Valoración mediante Monte Carlo

Bajo la medida neutral al riesgo, la dinámica utilizada para el activo es

$$dS_t = rS_t dt + \sigma S_t dW_t. \quad (30)$$

Por tanto, el precio terminal puede simularse mediante

$$S_T^{(i)} = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z_i \right], \quad Z_i \sim N(0, 1). \quad (31)$$

El estimador de Monte Carlo para una opción de compra europea es

$$\hat{C}_N = e^{-rT} \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0). \quad (32)$$

A medida que aumenta N , este estimador converge al valor esperado descontado del pago de la opción. En consecuencia, el valor obtenido por Monte Carlo puede compararse directamente con C_{BS} .

3.3. Error de la estimación

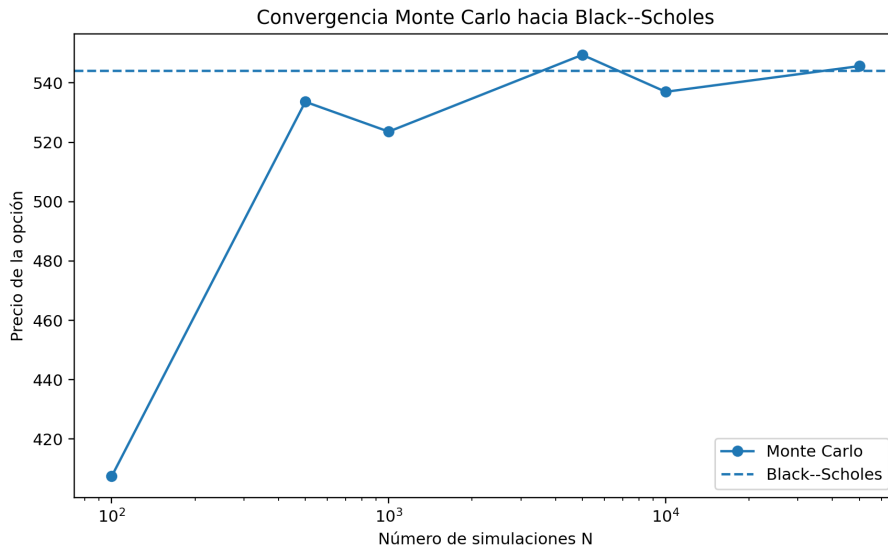


Figura 2: Convergencia de la estimación Monte Carlo hacia el valor analítico de Black-Scholes.

Para cuantificar la diferencia entre ambos procedimientos se utiliza el error relativo

$$\varepsilon_N = \frac{|\hat{C}_N - C_{BS}|}{|C_{BS}|}. \quad (33)$$

Esta comparación permite observar de forma práctica la relación entre el número de simulaciones y la precisión del resultado. El comportamiento esperado está relacionado con la tasa de convergencia de Monte Carlo,

$$\varepsilon_N = O(N^{-1/2}). \quad (34)$$

Por tanto, esta primera comparación cumple una función de validación numérica: se parte de un problema para el que existe una solución analítica y se comprueba que el método de simulación se aproxima a ella al aumentar el número de trayectorias.

Cuadro 1: Comparación entre la valoración mediante Monte Carlo y el valor analítico de Black–Scholes.

N	Monte Carlo	Black–Scholes	Error absoluto
100	407.431	544.067	136.636
500	533.650	544.067	10.418
1000	523.598	544.067	20.469
5000	549.465	544.067	5.398
10000	537.011	544.067	7.057
50000	545.675	544.067	1.607

Los resultados de la Tabla 1 muestran que, aunque el error no disminuye de forma estrictamente monótona debido al carácter aleatorio del método, la estimación obtenida con un mayor número de simulaciones se aproxima al valor analítico de Black–Scholes.

3.4. Diferencia respecto a la comparación con GARCH

Es importante separar esta primera comparación de la que se realiza posteriormente entre GBM y GARCH(1,1). Black–Scholes no se incorpora como un modelo alternativo para describir la serie histórica del S&P 500. En esta parte se utiliza como referencia analítica para validar una valoración obtenida mediante Monte Carlo. En cambio, GARCH(1,1) sí se emplea como una alternativa estadística para modelar los rendimientos y su varianza condicional. Por este motivo, las dos comparaciones responden a preguntas diferentes y se mantienen en capítulos separados.

4. Datos reales y preprocesamiento

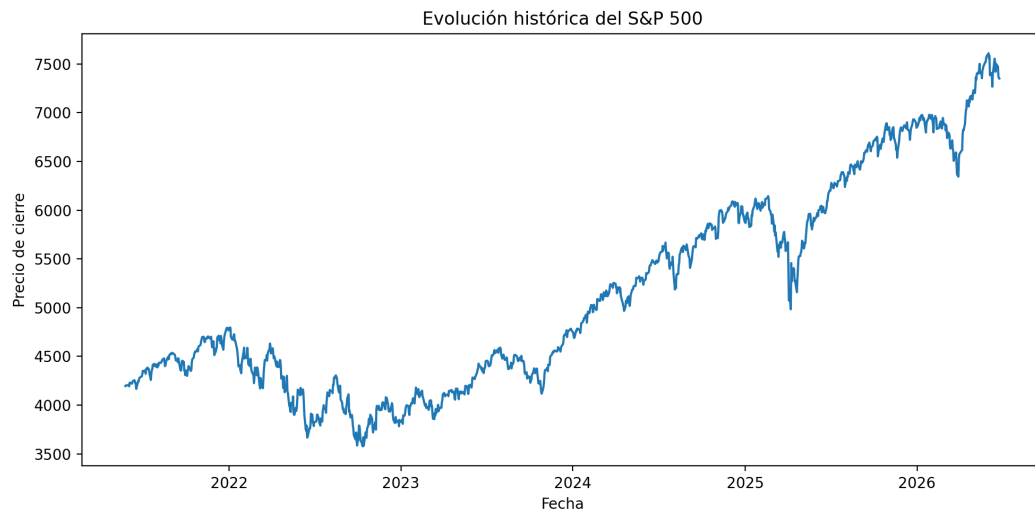


Figura 3: Evolución histórica del precio de cierre del S&P 500 utilizado en el estudio.

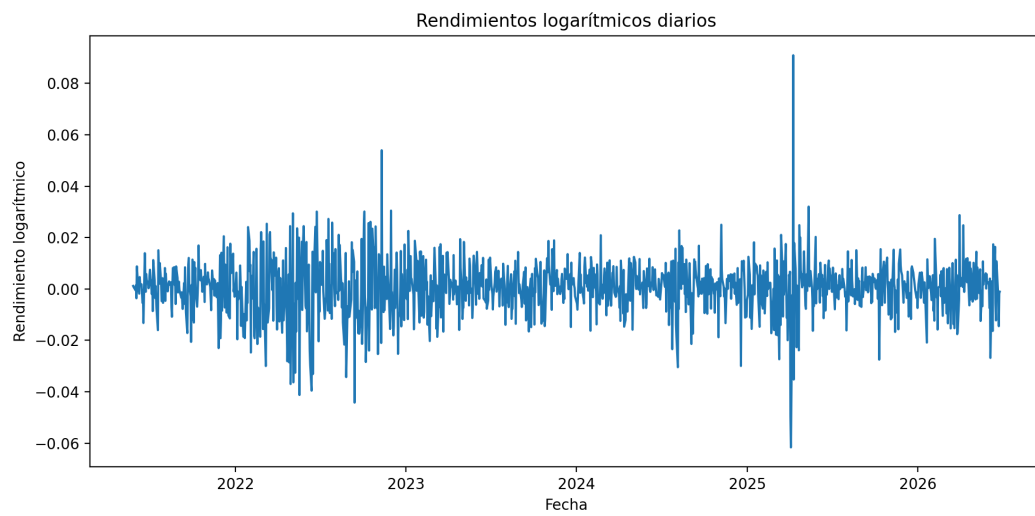


Figura 4: Evolución de los rendimientos logarítmicos diarios.

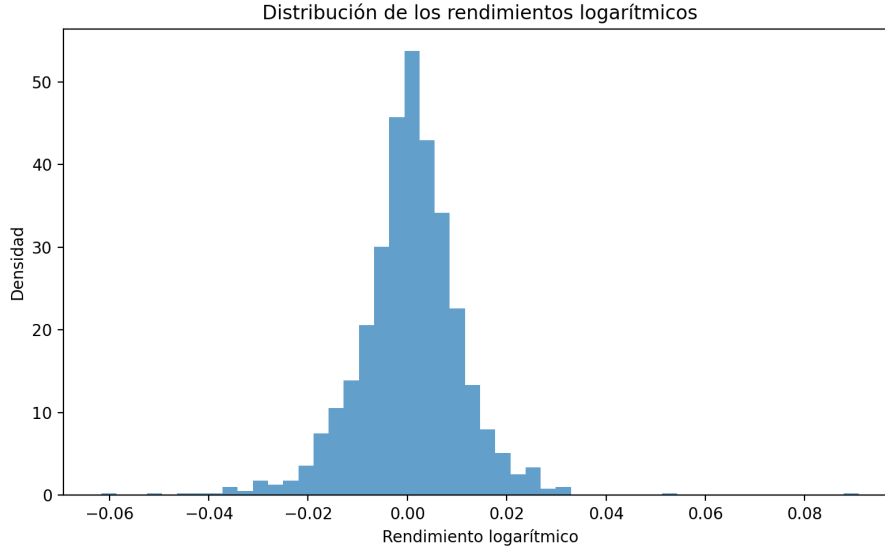


Figura 5: Distribución empírica de los rendimientos logarítmicos.

4.1. Calidad y preprocesamiento de los datos

Antes de realizar cualquier transformación o eliminación de observaciones, se comprobó la calidad del conjunto de datos original. El archivo contiene 1.276 observaciones correspondientes a precios diarios del índice S&P 500.

La comprobación inicial no detectó precios faltantes ni fechas duplicadas. Por tanto, el conjunto de datos original presenta 1.276 observaciones válidas en cuanto a estos criterios de calidad.

Cuadro 2: Comprobación de la calidad de los datos originales

Característica	Resultado
Observaciones originales	1.276
Precios faltantes	0
Fechas duplicadas	0

Una vez realizada esta comprobación, se procedió al preprocesamiento de los datos. Las fechas se transformaron al formato temporal correspondiente y los precios se convirtieron a valores numéricos. Posteriormente, se eliminaron las observaciones que pudieran contener valores no válidos y se ordenaron cronológicamente las observaciones, eliminando posibles fechas duplicadas.

4.2. Índice y variables utilizadas

La calibración y validación empírica se realizan utilizando datos históricos del índice S&P 500. El archivo empleado contiene las variables de fecha, precio, apertura, máximo,

mínimo, volumen y variación porcentual.

La variable principal utilizada para la calibración es el precio diario de cierre, ya que permite construir los rendimientos logarítmicos necesarios para la estimación de los parámetros de los modelos.

4.3. Período temporal y número de observaciones

La muestra utilizada contiene 1276 observaciones y comprende el período entre el 26 de mayo de 2021 y el 25 de junio de 2026. Antes de la calibración, la serie debe ordenarse cronológicamente y transformarse a un formato temporal adecuado.

4.4. Análisis de normalidad de los rendimientos

Con el objetivo de complementar el análisis gráfico de los rendimientos logarítmicos, se realizaron diversos contrastes estadísticos para evaluar la hipótesis de normalidad de la serie. En concreto, se aplicaron las pruebas de Lilliefors, Shapiro–Wilk y Anderson–Darling sobre los rendimientos logarítmicos diarios obtenidos a partir de los precios de cierre.

Las pruebas de Lilliefors y Shapiro–Wilk permiten contrastar la hipótesis nula de que los datos proceden de una distribución normal. Para un nivel de significación del 5 %, un valor p inferior a 0.05 conduce al rechazo de dicha hipótesis. Por su parte, la prueba de Anderson–Darling se interpreta comparando el estadístico obtenido con el valor crítico correspondiente al nivel de significación considerado.

Los resultados obtenidos se muestran en la Tabla 3.

Cuadro 3: Resultados de las pruebas de normalidad aplicadas a los rendimientos logarítmicos

Prueba	Estadístico	p -valor	V.C. 5 %	Decisión
Lilliefors	0.071337	1.00×10^{-3}	–	Rechazar
Shapiro–Wilk	0.945841	3.19×10^{-21}	–	Rechazar
Anderson–Darling	11.037590	–	0.785	Rechazar

Los resultados muestran que las tres pruebas conducen al rechazo de la hipótesis de normalidad al nivel de significación del 5 %. En particular, los valores p obtenidos mediante las pruebas de Lilliefors y Shapiro–Wilk son inferiores a 0.05. Asimismo, en la prueba de Anderson–Darling el estadístico obtenido, igual a 11.037590, es muy superior al valor crítico de 0.785 correspondiente al nivel de significación del 5 %.

Por tanto, los resultados empíricos indican que los rendimientos logarítmicos del índice S&P 500 analizados no siguen estrictamente una distribución normal. Este comportamiento es coherente con una característica habitual de las series financieras, que

pueden presentar observaciones extremas y desviaciones respecto a la distribución gaussiana, especialmente en las colas de la distribución.

Este resultado no invalida la utilización del Movimiento Browniano Geométrico como modelo de referencia, pero permite identificar una de sus principales simplificaciones al aplicarlo a datos reales. El GBM supone innovaciones gaussianas y volatilidad constante, mientras que la evidencia empírica muestra que la distribución observada presenta desviaciones respecto a estas hipótesis. Esta limitación constituye una motivación adicional para incorporar posteriormente el modelo GARCH(1,1), que permite representar una volatilidad condicional variable en el tiempo.

Nota: En la prueba de Anderson–Darling no se muestra un p -valor, ya que la decisión se realiza comparando el estadístico de contraste con los valores críticos correspondientes a cada nivel de significación.

4.5. Días no bursátiles y datos faltantes

La serie contiene únicamente sesiones de negociación. Por tanto, los fines de semana y los días en los que el mercado permanece cerrado no se incorporan como observaciones artificiales.

La comprobación inicial de calidad, realizada antes del preprocesamiento, mostró que no existen precios faltantes ni fechas duplicadas en el archivo original. Por este motivo, no fue necesario aplicar procedimientos de imputación.

Es importante distinguir entre días laborables y días bursátiles. Para la validación temporal se utilizan directamente las fechas reales del conjunto de validación y no se genera una secuencia artificial mediante todos los días laborables, ya que esta podría incluir días festivos en los que no existió cotización.

4.6. División entrenamiento-validación

La muestra se divide cronológicamente en un 80 % para calibración y un 20 % para validación fuera de muestra. De este modo, se utilizan 1020 observaciones en entrenamiento y 256 observaciones en validación.

La última observación del conjunto de entrenamiento corresponde al 17 de junio de 2025 y la primera del conjunto de validación al 18 de junio de 2025.

Esta separación evita que la información futura se utilice durante la estimación de parámetros.

5. Metodología e implementación del GBM

5.1. Calibración de μ y σ

A partir de los precios históricos se construyen los rendimientos

$$r_i = \log \left(\frac{S_i}{S_{i-1}} \right). \quad (35)$$

Si $\bar{r}$ es la media muestral y s_r^2 la varianza muestral, la volatilidad anualizada se estima mediante

$$\hat{\sigma} = \sqrt{\frac{s_r^2}{\Delta t}}. \quad (36)$$

Para datos diarios y 252 sesiones anuales,

$$\hat{\sigma}_{\text{anual}} = \sqrt{252} s_r. \quad (37)$$

La estimación de la deriva se obtiene de

$$\hat{\mu} = \frac{\bar{r}}{\Delta t} + \frac{1}{2} \hat{\sigma}^2. \quad (38)$$

5.2. Discretización temporal

Si el horizonte temporal es T y se utilizan M pasos,

$$\Delta t = \frac{T}{M}. \quad (39)$$

La actualización exacta del GBM es

$$S_{t+\Delta t} = S_t \exp \left[\left(\mu - \frac{1}{2} \sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \right], \quad (40)$$

donde $Z \sim N(0, 1)$.

5.3. Estadísticos y validación

Para las simulaciones se calculan la media, mediana, varianza, percentiles y bandas predictivas. La validación fuera de muestra se complementa con las métricas

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (S_t - \hat{S}_t)^2}, \quad (41)$$

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |S_t - \hat{S}_t|, \quad (42)$$

y

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^n \left| \frac{S_t - \hat{S}_t}{S_t} \right|. \quad (43)$$

6. Validación del GBM con datos simulados

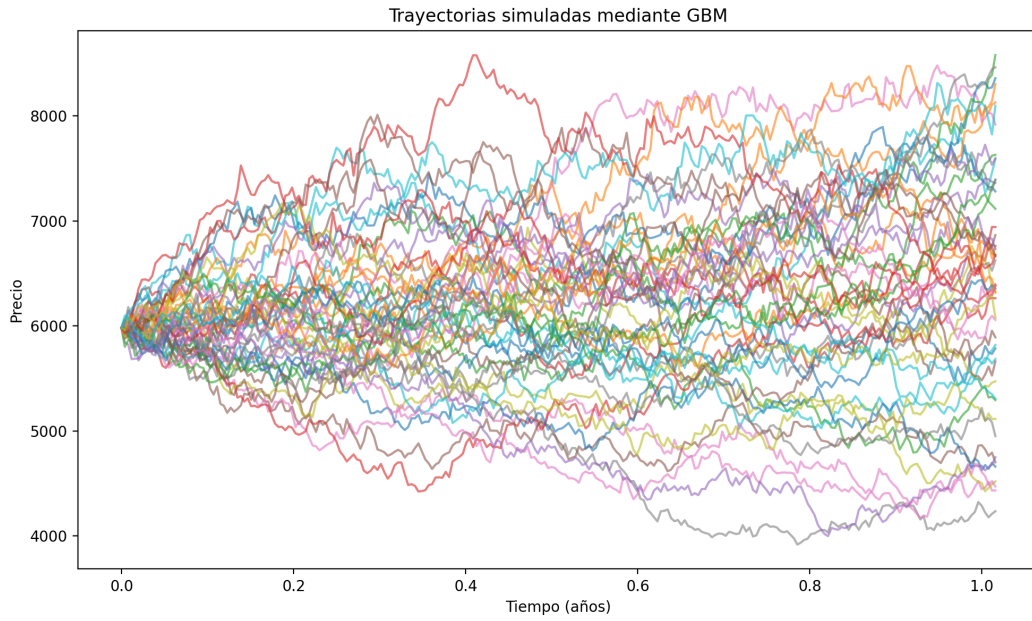


Figura 6: Ejemplo de trayectorias simuladas mediante el movimiento browniano geométrico.

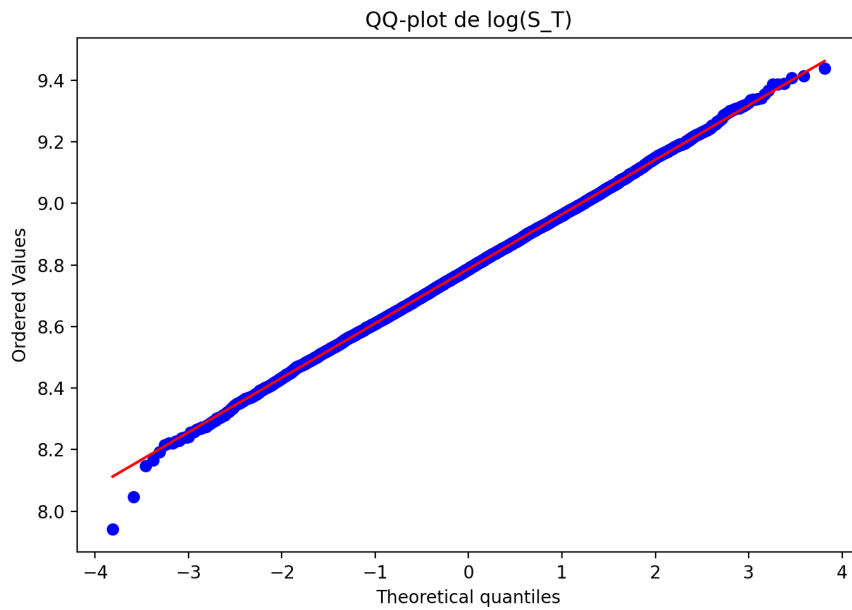


Figura 7: QQ-plot de $\log(S_T)$ frente a una distribución normal.

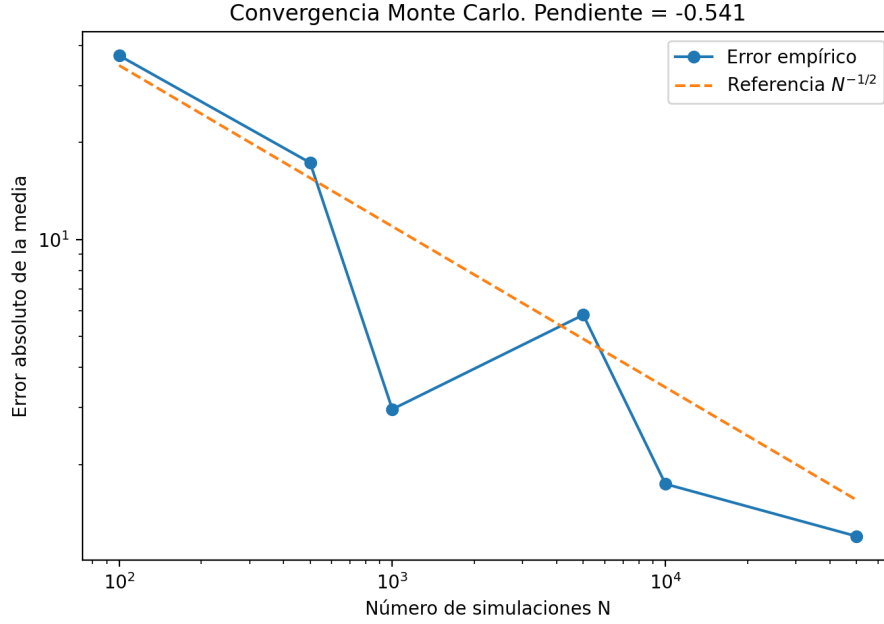


Figura 8: Convergencia empírica del método de Monte Carlo.

La implementación se valida comparando los resultados obtenidos mediante simulación con las propiedades analíticas del modelo. En particular, se comparan la media y la varianza del precio terminal simulado con sus correspondientes valores teóricos. Asimismo, se analiza la distribución de $\log(S_T)$ mediante un gráfico QQ, con el objetivo de comprobar visualmente su compatibilidad con una distribución normal.

Los resultados obtenidos permiten comprobar que la simulación reproduce las propiedades teóricas del Movimiento Browniano Geométrico. Las diferencias entre los valores simulados y los valores teóricos se deben al carácter aleatorio de la simulación y disminuyen, en términos generales, al aumentar el número de simulaciones.

También se analiza la convergencia empírica del método de Monte Carlo utilizando diferentes valores de N . En una representación logarítmica del error frente al número de simulaciones, se espera observar un comportamiento compatible con una pendiente aproximada de $-1/2$, de acuerdo con el orden teórico de convergencia del método de Monte Carlo.

Cuadro 4: Validación de los momentos teóricos del GBM frente a los resultados simulados.

Estadístico	Teórico	Simulado	Error relativo (%)
Media	6646.697	6655.203	0.128
Varianza	1.448×10^6	1.412×10^6	2.472

La Tabla 4 confirma que los momentos obtenidos mediante simulación son coherentes con las expresiones analíticas del GBM. El error relativo es reducido tanto para la media

como para la varianza.

Cuadro 5: Convergencia empírica del método de Monte Carlo.

N	Error absoluto
100	37.2511
500	17.3103
1000	2.9682
5000	5.8286
10000	1.7393
50000	1.1971

La pendiente estimada en la regresión logarítmica del error frente al número de simulaciones fue -0.541225 , valor próximo al orden teórico $-1/2$. Este resultado proporciona evidencia numérica de la tasa de convergencia $O(N^{-1/2})$ característica del método de Monte Carlo.

7. Análisis de sensibilidad del GBM

7.1. Sensibilidad respecto a μ

Se consideran diferentes valores de la deriva manteniendo constante la volatilidad. La esperanza del precio terminal es

$$\mathbb{E}[S_T] = S_0 e^{\mu T}. \quad (44)$$

Por tanto, un aumento de μ incrementa el nivel esperado del precio. Además, la varianza terminal también depende de la deriva:

$$\text{Var}(S_T) = S_0^2 e^{2\mu T} (e^{\sigma^2 T} - 1). \quad (45)$$

En consecuencia, no es correcto afirmar que μ afecta exclusivamente a la media. Manteniendo σ fijo, un aumento de μ también incrementa la varianza absoluta del precio terminal.

Cuadro 6: Sensibilidad de la media y la varianza terminal ante variaciones de μ .

μ	Media simulada	Varianza simulada
0.02	4287.65	609111.96
0.05	4418.22	646777.34
0.08	4552.78	686771.82
0.12	4738.58	743971.03

Los resultados muestran que un incremento de μ eleva la media del precio terminal y, manteniendo constante σ , también aumenta la varianza absoluta del proceso, en concordancia con la expresión teórica.

7.2. Sensibilidad respecto a σ

La volatilidad controla directamente la dispersión del proceso. Según la expresión teórica,

$$\text{Var}(S_T) = S_0^2 e^{2\mu T} (e^{\sigma^2 T} - 1), \quad (46)$$

la dependencia respecto a σ es no lineal. Al aumentar σ , la varianza crece a través del término $e^{\sigma^2 T} - 1$.

El análisis experimental compara la varianza simulada con la varianza teórica para diferentes valores de μ y σ .

Cuadro 7: Sensibilidad de la media y la varianza terminal ante variaciones de σ .

σ	Media simulada	Varianza simulada
0.10	4549.48	208661.50
0.18	4552.78	686771.82
0.25	4555.81	1350348.00
0.35	4560.48	2743578.00

A diferencia de la deriva, las variaciones de σ apenas modifican la media simulada, pero producen un incremento muy significativo de la varianza. Esto pone de manifiesto el papel central de la volatilidad en la dispersión de las trayectorias y motiva el uso de modelos con volatilidad variable, como GARCH.

7.3. Metodología del análisis

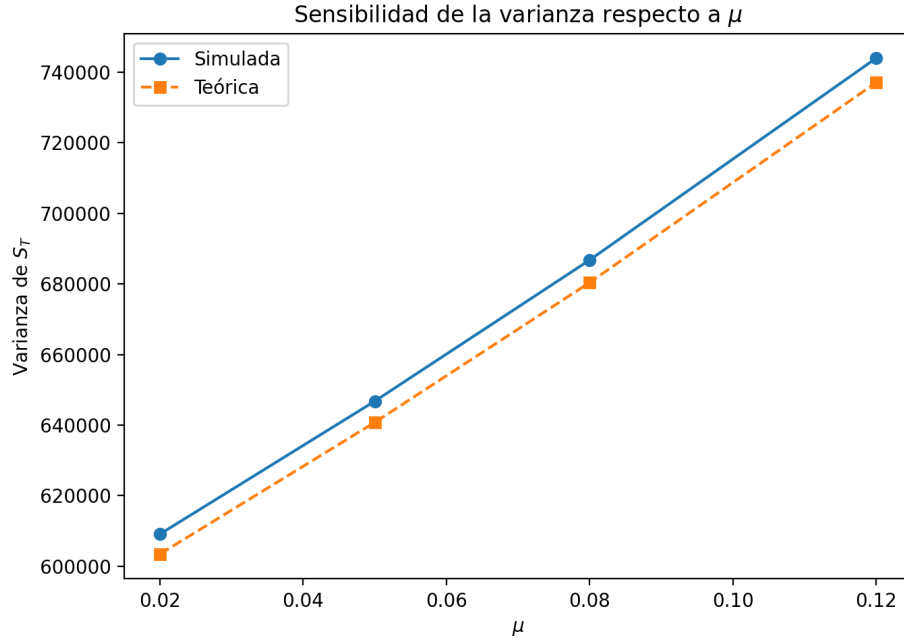


Figura 9: Sensibilidad de la varianza terminal respecto a diferentes valores de μ .

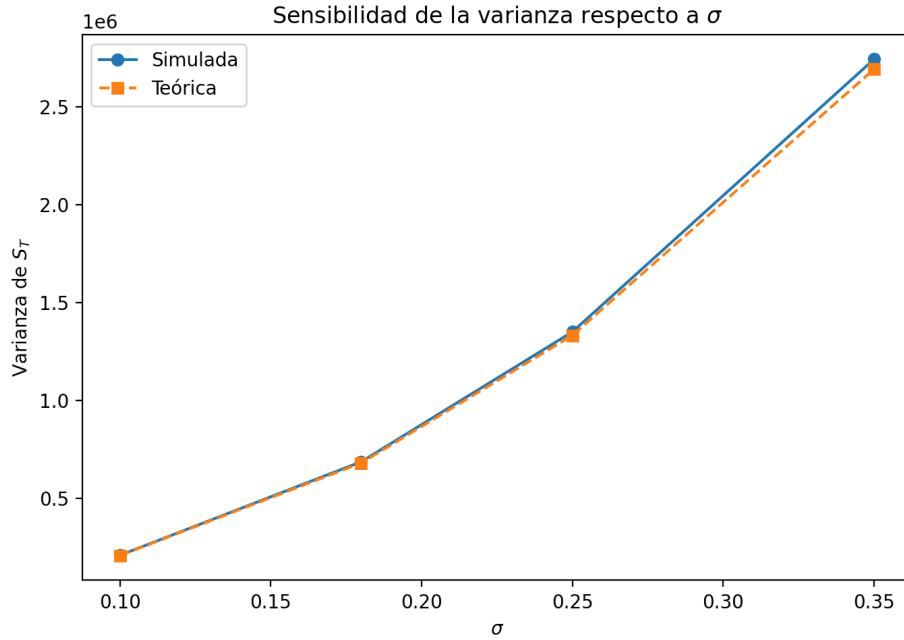


Figura 10: Sensibilidad de la varianza terminal respecto a diferentes valores de σ .

Para cada combinación de parámetros se generan N trayectorias independientes y se calcula

$$\widehat{\text{Var}}(S_T) = \frac{1}{N-1} \sum_{i=1}^N \left(S_T^{(i)} - \bar{S}_T \right)^2. \quad (47)$$

Los resultados se presentan mediante dos tablas y dos representaciones gráficas, diferenciando el efecto de μ y de σ sobre la media y la varianza del precio terminal.

8. Modelo GARCH(1,1)

Los modelos de heterocedasticidad condicional tienen su origen en el modelo ARCH propuesto por Engle [6]. Posteriormente, Bollerslev extendió esta formulación mediante el modelo GARCH [5].

8.1. Motivación

Una de las principales limitaciones del GBM es la hipótesis de volatilidad constante. En datos financieros reales es frecuente observar períodos de alta y baja volatilidad, así como agrupamiento temporal de episodios de elevada variabilidad.

El modelo GARCH(1,1) permite representar una varianza condicional que cambia con el tiempo y puede calibrarse utilizando los mismos rendimientos logarítmicos empleados en el análisis del GBM.

8.2. Definición

Sea r_t el rendimiento logarítmico. Un modelo básico puede escribirse como

$$r_t = m + \varepsilon_t, \quad (48)$$

donde

$$\varepsilon_t = \sigma_t z_t, \quad (49)$$

con z_t una innovación de media cero y varianza unidad.

La varianza condicional sigue la dinámica

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2. \quad (50)$$

Los parámetros tienen la siguiente interpretación:

- ω : nivel base de la varianza condicional.
- α : sensibilidad frente a innovaciones recientes.

- β : persistencia de la volatilidad.

Para garantizar una varianza positiva se requiere habitualmente

$$\omega > 0, \quad \alpha \geq 0, \quad \beta \geq 0. \quad (51)$$

Una condición habitual para la estacionariedad de segundo orden es

$$\alpha + \beta < 1. \quad (52)$$

8.3. Calibración

La calibración de GARCH(1,1) se realiza sobre los rendimientos del conjunto de entrenamiento. Los parámetros se estiman mediante máxima verosimilitud, utilizando una distribución condicional especificada para las innovaciones.

Para mantener la comparabilidad con el GBM, deben utilizarse exactamente los mismos datos de entrenamiento y reservar el mismo período para la validación.

Cuadro 8: Parámetros estimados del modelo GARCH(1,1).

Parámetro	Estimación
μ diario	0.000709
ω	0.000003
α	0.105301
β	0.871918
$\alpha + \beta$	0.977219

Destaca que $\alpha + \beta = 0.977219$ es un valor próximo a uno, lo que indica una elevada persistencia de la volatilidad condicional. Esta característica no puede ser recogida por el GBM estándar, que supone una volatilidad constante.

8.4. Simulación

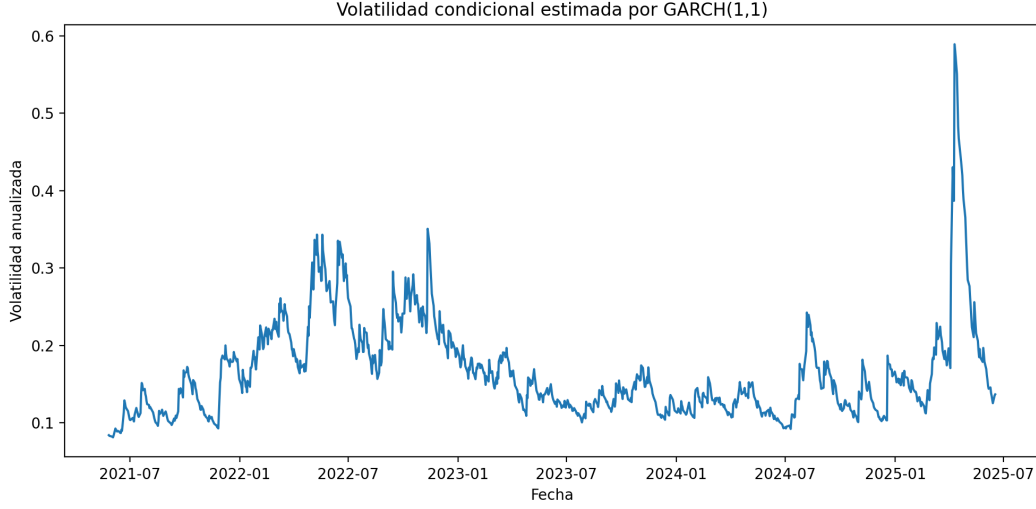


Figura 11: Evolución de la volatilidad condicional estimada mediante el modelo GARCH(1,1).

Una vez estimados los parámetros, se generan innovaciones y actualizar recursivamente la varianza mediante (50). Los rendimientos simulados se acumulan para reconstruir los precios:

$$S_t = S_0 \exp \left(\sum_{i=1}^t r_i \right). \quad (53)$$

9. Comparación entre GBM y GARCH(1,1)

9.1. Diseño experimental

La comparación se realiza bajo condiciones equivalentes:

1. mismo índice financiero;
2. mismo período histórico;
3. misma división entrenamiento-validación;
4. mismo precio inicial para la reconstrucción fuera de muestra;
5. mismo horizonte temporal;
6. métricas de evaluación comunes.

9.2. Comparación conceptual

Cuadro 9: Comparación conceptual entre GBM y GARCH(1,1).

Característica	GBM	GARCH(1,1)
Variable principal	Precio	Rendimiento
Volatilidad	Constante	Variable en el tiempo
Clustering de volatilidad	No	Sí
Calibración	Momentos de rendimientos	Máxima verosimilitud
Complejidad	Baja	Moderada
Simulación	Directa	Recursiva
Datos históricos	Estimación de μ, σ	Estimación de parámetros condicionales

9.3. Comparación de volatilidad

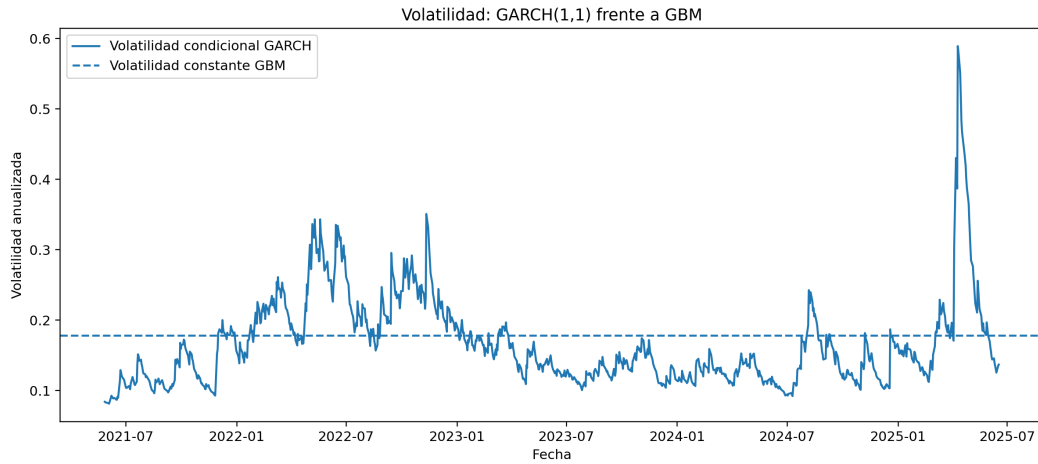


Figura 12: Comparación entre la volatilidad condicional estimada por GARCH(1,1) y la volatilidad constante del GBM.

En el GBM la volatilidad utilizada durante la simulación es constante:

$$\sigma_t = \sigma. \quad (54)$$

En GARCH, la volatilidad condicional cambia en cada instante:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2. \quad (55)$$

Por tanto, una comparación relevante consiste en representar conjuntamente los rendimientos reales, la volatilidad condicional estimada por GARCH y la volatilidad

constante equivalente utilizada por el GBM.

9.4. Comparación fuera de muestra

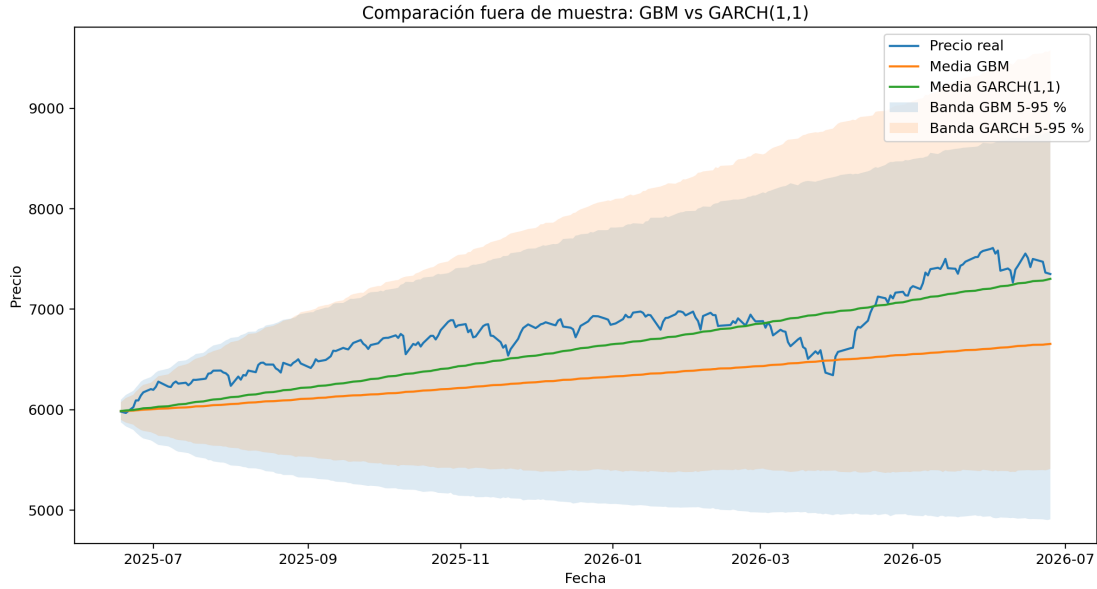


Figura 13: Comparación fuera de muestra entre el precio real y las simulaciones obtenidas con GBM y GARCH(1,1).

Para ambos modelos se calcularán RMSE, MAE y MAPE. Además, cuando se generen múltiples trayectorias, se estudiará la cobertura de intervalos predictivos.

Los resultados se presentarán en una tabla final:

Cuadro 10: Resultados definitivos de la comparación fuera de muestra entre GBM y GARCH(1,1).

Métrica	GBM	GARCH(1,1)
RMSE	525.261	261.771
MAE	480.577	235.262
MAPE (%)	6.951	3.478
Cobertura 5-95 % (%)	100.000	100.000

Los resultados muestran que GARCH(1,1) presenta menores errores que el GBM en las tres métricas de precisión consideradas. En particular, el MAPE se reduce aproximadamente del 6.95 % al 3.48 %, lo que indica un mejor comportamiento en la comparación fuera de muestra realizada.

10. Discusión y limitaciones

El GBM ofrece una estructura matemática sencilla, una solución analítica y una implementación eficiente mediante Monte Carlo. Además, permite obtener expresiones cerradas para la esperanza y la varianza del precio terminal.

Sin embargo, supone una volatilidad constante y una distribución normal para los rendimientos logarítmicos. Estas hipótesis pueden resultar restrictivas en series financieras reales.

El modelo GARCH(1,1) permite relajar la hipótesis de volatilidad constante al introducir una dinámica para la varianza condicional. Su incorporación no sustituye necesariamente al GBM en todos los contextos, ya que ambos modelos describen aspectos diferentes: el GBM proporciona una dinámica directa para el precio, mientras que GARCH se centra en la evolución de los rendimientos y de su volatilidad condicional.

Por tanto, el objetivo de la comparación no debe ser afirmar de forma general que uno de los modelos es siempre superior, sino identificar en qué aspectos cada uno representa mejor los datos y qué coste adicional introduce.

11. Conclusiones

En este trabajo se ha desarrollado una metodología para el estudio de modelos estocásticos aplicados a la simulación financiera. El movimiento browniano geométrico proporciona una base matemática adecuada para introducir procesos estocásticos, cálculo de Itô y métodos de Monte Carlo.

La validación de la implementación permite comprobar la coherencia entre las simulaciones y las propiedades analíticas del GBM. La calibración con datos reales permite trasladar el modelo desde un entorno puramente teórico a un problema empírico.

El análisis de sensibilidad muestra que tanto la deriva como la volatilidad influyen en la distribución del precio terminal. En particular, la volatilidad presenta una relación no lineal con la varianza, mientras que la deriva también afecta a la varianza absoluta mediante el nivel esperado del proceso.

La incorporación del modelo GARCH(1,1) amplía el estudio hacia una volatilidad variable en el tiempo. La comparación final entre ambos modelos permite analizar empíricamente cómo afecta esta diferencia estructural a la descripción y validación de los datos históricos.

Como trabajo futuro, podrían estudiarse extensiones con distribuciones de innovaciones no normales, modelos EGARCH o GJR-GARCH, volatilidad estocástica o modelos con saltos.

Referencias

- [1] Hull, J. C. (2021). *Options, Futures and Other Derivatives*. Pearson.
- [2] Shreve, S. E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer.
- [3] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer.
- [4] Black, F. y Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [5] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- [6] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

A. Código fuente

El código fuente utilizado para la realización de los experimentos numéricos se incluye en este anexo. Adicionalmente, con el objetivo de facilitar la reproducibilidad y consulta del proyecto completo, el código y los ficheros necesarios para su ejecución se encuentran disponibles en el siguiente repositorio:

<https://github.com/el-Tito/simulacion-precios-activos-monte-carlo>

A.1. Implementación del movimiento browniano geométrico

Listing 1: Código fuente del fichero `simulador_gbm.py`

```
"""
simulador_gbm.py
=====
Funciones y clase para simular un Movimiento Browniano
Geométrico (GBM).

Modelo:
    dS_t = mu S_t dt + sigma S_t dW_t

Solución exacta discretizada:
    S_{t+dt} = S_t exp((mu - 0.5 sigma^2) dt
                      + sigma sqrt(dt) Z)
```

```

"""

from dataclasses import dataclass
import numpy as np

@dataclass
class SimuladorGBM:
    S0: float
    mu: float
    sigma: float
    T: float = 1.0
    pasos: int = 252

    @property
    def dt(self):
        return self.T / self.pasos

    def simular(self, N=10000, seed=None):
        """
        Genera N trayectorias independientes del GBM.

        Devuelve
        -----
        tiempos : ndarray, shape (pasos + 1,)
        S : ndarray, shape (N, pasos + 1)
        """
        rng = np.random.default_rng(seed)
        Z = rng.standard_normal((N, self.pasos))

        incrementos_log = (
            (self.mu - 0.5 * self.sigma**2) * self.dt
            + self.sigma * np.sqrt(self.dt) * Z
        )

        log_relativos = np.cumsum(incrementos_log, axis=1)

        S = np.empty((N, self.pasos + 1))
        S[:, 0] = self.S0
        S[:, 1:] = self.S0 * np.exp(log_relativos)

```

```

    tiempos = np.linspace(0.0, self.T, self.pasos + 1)
    return tiempos, S

def precio_terminal_directo(self, N=10000, seed=None):
    """Simula directamente S_T usando la soluci n cerrada."
    """
    rng = np.random.default_rng(seed)
    Z = rng.standard_normal(N)

    ST = self.S0 * np.exp(
        (self.mu - 0.5 * self.sigma**2) * self.T
        + self.sigma * np.sqrt(self.T) * Z
    )
    return ST

def esperanza_teorica(self, t=None):
    """E[S_t] = S0 exp(mu t)."""
    if t is None:
        t = self.T
    return self.S0 * np.exp(self.mu * np.asarray(t))

def varianza_teorica(self, t=None):
    """
    Var(S_t) =
    S0^2 exp(2 mu t) [exp(sigma^2 t) - 1].
    """
    if t is None:
        t = self.T
    t = np.asarray(t)
    return (
        self.S0**2
        * np.exp(2.0 * self.mu * t)
        * (np.exp(self.sigma**2 * t) - 1.0)
    )

@staticmethod
def estadisticos_terminales(S):
    """Calcula estad sticos emp ricos del precio terminal.
    """
    ST = S[:, -1]
    return {

```

```

        "media": float(np.mean(ST)),
        "mediana": float(np.median(ST)),
        "varianza": float(np.var(ST, ddof=1)),
        "desviacion_tipica": float(np.std(ST, ddof=1)),
        "p5": float(np.percentile(ST, 5)),
        "p50": float(np.percentile(ST, 50)),
        "p95": float(np.percentile(ST, 95)),
    }

def calibrar_gbm(precios, sesiones_anuales=252):
    """
    Calibra mu y sigma a partir de precios hist ricos.

    Para rendimientos logar tmicos diarios r:
        sigma = sqrt(252) * std(r)
        mu = 252 * mean(r) + 0.5 * sigma^2
    """
    precios = np.asarray(precios, dtype=float)
    if len(precios) < 2:
        raise ValueError("Se necesitan al menos dos precios.")

    r = np.diff(np.log(precios))
    sigma = np.sqrt(sesiones_anuales) * np.std(r, ddof=1)
    mu = sesiones_anuales * np.mean(r) + 0.5 * sigma**2

    return {
        "mu": float(mu),
        "sigma": float(sigma),
        "rendimientos": r,
    }

```

A.2. Análisis y validación del GBM

Listing 2: Código fuente del fichero analisis_gbm.py

```

"""
analisis_gbm.py
=====
Calibraci n, validaci n, convergencia Monte Carlo y an lisis
de sensibilidad

```

```

del GBM utilizando datos_sp500.csv.
"""

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from statsmodels.stats.diagnostic import lilliefors

from simulador_gbm import SimuladorGBM, calibrar_gbm

BASE = Path(__file__).resolve().parent
DATA_PATH = BASE.parent / "datos" / "datos_sp500.csv"
RESULTADOS = BASE.parent / "resultados"
FIGURAS = BASE.parent / "figuras"
RESULTADOS.mkdir(exist_ok=True)
FIGURAS.mkdir(exist_ok=True)

SESIONES_ANUALES = 252
SEED = 42
N_SIMULACIONES = 10_000

def cargar_datos(path=DATA_PATH):
    """
        Carga el CSV de Investing.com.

        El archivo puede contener precios con separadores de
        miles o comas.
        La función intenta convertir la columna Price de forma
        robusta y
        comprueba la calidad de los datos originales antes de
        realizar la limpieza.
    """

    # Cargar datos originales
    df = pd.read_csv(path)

    if "Date" not in df.columns or "Price" not in df.columns:

```

```

        raise ValueError(
            "El CSV debe contener las columnas 'Date' y 'Price'."
        )

# N mero de observaciones originales
observaciones_originales = len(df)

# Conversi n de fechas
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

# Conversi n robusta del precio, eliminando separadores de miles
precio = (
    df["Price"]
    .astype(str)
    .str.replace(",", "", regex=False)
    .str.replace(" ", "", regex=False)
)
df["Price"] = pd.to_numeric(precio, errors="coerce")

# Comprobaci n de calidad de los datos ORIGINALES
faltantes_originales = int(
    df["Price"].isna().sum()
)

duplicados_originales = int(
    df["Date"].duplicated().sum()
)

# Limpieza de los datos
df = (
    df.dropna(subset=["Date", "Price"])
    .sort_values("Date")
    .drop_duplicates(subset="Date")
    .reset_index(drop=True)
)

return (
    df,
    observaciones_originales,

```

```

        faltantes_originales,
        duplicados_originales,
    )

def descripcion_datos(df):
    """Genera estadísticas básicas y comprobaciones de calidad
    ."""
    resumen = {
        "fecha_inicial": df["Date"].min(),
        "fecha_final": df["Date"].max(),
        "n_observaciones": len(df),
        "fechas_duplicadas": int(df["Date"].duplicated().sum()),
        "faltantes_price": int(df["Price"].isna().sum()),
    }

    resumen_df = pd.DataFrame(
        [{"variable": k, "valor": v} for k, v in resumen.items()]
    )
    resumen_df.to_csv(RESULTADOS / "descripcion_datos.csv",
        index=False)
    return resumen

def dividir_train_test(df, proporcion_train=0.80):
    """
        Divide la serie temporal respetando el orden cronológico
        .

        El 80% inicial se utiliza para calibración y el 20%
        final
        para validación fuera de muestra.
    """

    n_train = int(len(df) * proporcion_train)
    train = df.iloc[:n_train].copy()
    test = df.iloc[n_train:].copy()
    return train, test

def metricas(y_real, y_pred):

```

```

"""Calcula RMSE, MAE y MAPE."""
y_real = np.asarray(y_real, dtype=float)
y_pred = np.asarray(y_pred, dtype=float)

rmse = np.sqrt(np.mean((y_real - y_pred) ** 2))
mae = np.mean(np.abs(y_real - y_pred))

mascara = y_real != 0
mape = 100.0 * np.mean(
    np.abs((y_real[mascara] - y_pred[mascara]) / y_real[
        mascara])
)

return {"RMSE": rmse, "MAE": mae, "MAPE": mape}

def graficas_descriptivas(df):
    """
        Genera gr ficos descriptivos de precios y rendimientos.
    """

    precios = df["Price"].to_numpy(dtype=float)
    fechas = df["Date"]
    rendimientos = np.diff(np.log(precios))
    fechas_r = fechas.iloc[1:]

    # -----
    # Evoluci n hist rica del precio
    # -----

    plt.figure(figsize=(10, 5))
    plt.plot(fechas, precios)

    plt.xlabel("Fecha")
    plt.ylabel("Precio de cierre")
    plt.title("Evoluci n hist rica del S&P 500")

    plt.tight_layout()

    plt.savefig(FIGURAS / "precio_historico.png", dpi=200)

```

```

plt.close()

# -----
# Rendimientos logar tmicos
# -----
plt.figure(figsize=(10, 5))
plt.plot(fechas_r, rendimientos)

plt.xlabel("Fecha")
plt.ylabel("Rendimiento logar tmico")
plt.title("Rendimientos logar tmicos diarios")

plt.tight_layout()

plt.savefig(FIGURAS / "rendimientos_logaritmicos.png", dpi
            =200)

plt.close()

# -----
# Histograma de rendimientos
# -----
plt.figure(figsize=(8, 5))

plt.hist(rendimientos, bins=50, density=True, alpha=0.7)

plt.xlabel("Rendimiento logar tmico")
plt.ylabel("Densidad")
plt.title("Distribuci n de los rendimientos logar tmicos")

plt.tight_layout()

plt.savefig(FIGURAS / "histograma_rendimientos.png", dpi
            =200); plt.close()

# =====
# PRUEBAS DE NORMALIDAD
# =====
def pruebas_normalidad(df):
    """

```

Realiza pruebas estadísticas de normalidad sobre los rendimientos logarítmicos diarios.

Se aplican:

- Kolmogorov-Smirnov
- Shapiro-Wilk
- Anderson-Darling

Los resultados se almacenan en:

resultados/pruebas_normalidad.csv

"""

```
precios = df["Price"].to_numpy(dtype=float)
```

```
rendimientos = np.diff(  
    np.log(precios)  
)
```

```
# =====  
# LILLIEFORS  
# =====
```

```
lillie_stat, lillie_pvalue = lilliefors(  
    rendimientos,  
    dist="norm"  
)
```

```
# =====  
# SHAPIRO-WILK  
# =====
```

```
shapiro_stat, shapiro_pvalue = stats.shapiro(  
    rendimientos  
)
```

```
# =====  
# ANDERSON-DARLING  
# =====
```

```
anderson = stats.anderson(  

```

```

rendimientos,
dist="norm"
)

# Obtener el valor cr tico correspondiente al 5 %
indice_5 = np.where(
    anderson.significance_level == 5.0
)[0][0]

valor_critico_5 = (
    anderson.critical_values[indice_5]
)

if anderson.statistic > valor_critico_5:
    decision_anderson = "Rechazar normalidad"
else:
    decision_anderson = "No rechazar normalidad"

# =====
# DECISIONES
# =====

decision_lilliefors = (
    "Rechazar normalidad"
    if lillie_pvalue < 0.05
    else "No rechazar normalidad"
)

decision_shapiro = (
    "Rechazar normalidad"
    if shapiro_pvalue < 0.05
    else "No rechazar normalidad"
)

# =====
# TABLA DE RESULTADOS
# =====

tabla = pd.DataFrame(
    {
        "Prueba": [

```

```

        "Lilliefors",
        "Shapiro-Wilk",
        "Anderson-Darling",
    ],

    "Estadistico": [
        lillie_stat,
        shapiro_stat,
        anderson.statistic,
    ],

    "p_valor": [
        lillie_pvalue,
        shapiro_pvalue,
        np.nan,
    ],

    "Valor_critico_5pct": [
        np.nan,
        np.nan,
        valor_critico_5,
    ],

    "Decision_5pct": [
        decision_lilliefors,
        decision_shapiro,
        decision_anderson,
    ],
}

)

tabla.to_csv(
    RESULTADOS / "pruebas_normalidad.csv",
    index=False
)

return tabla

def graficar_trayectorias(simulador, N=50):
    """Genera una muestra de trayectorias simuladas del GBM."""
    tiempos, S = simulador.simular(N=N, seed=SEED)

```

```

plt.figure(figsize=(10, 6))
for trayectoria in S: plt.plot(tiempos, trayectoria, alpha
    =0.6)
plt.xlabel("Tiempo (a os)")
plt.ylabel("Precio")
plt.title("Trayectorias simuladas mediante GBM")

plt.tight_layout()

plt.savefig(FIGURAS / "trayectorias_gbm.png", dpi=200)

plt.close()

# =====
# VALIDACION DE DISTRIBUCION
# =====
def validar_distribucion(simulador, N=N_SIMULACIONES):
    """Compara S_T simulado con sus momentos teóricos."""
    _, S = simulador.simular(N=N, seed=SEED)
    ST = S[:, -1]
    log_ST = np.log(ST)

    media_teo = simulador.esperanza_teorica()
    var_teo = simulador.varianza_teorica()

    media_sim = np.mean(ST)
    var_sim = np.var(ST, ddof=1)

    tabla = pd.DataFrame(
        {
            "estadistico": ["Media", "Varianza"],
            "teorico": [media_teo, var_teo],
            "simulado": [media_sim, var_sim],
        }
    )
    tabla["error_relativo_pct"] = (
        np.abs(tabla["simulado"] - tabla["teorico"])
        / np.abs(tabla["teorico"])
        * 100
    )
    tabla.to_csv(RESULTADOS / "validacion_gbm.csv", index=False)

```

```

# QQ-plot de log(S_T)
plt.figure(figsize=(7, 5))
stats.probplot(log_ST, dist="norm", plot=plt)
plt.title("QQ-plot de log(S_T)")
plt.tight_layout()
plt.savefig(FIGURAS / "qqplot_log_ST.png", dpi=200)
plt.close()

return tabla

# =====
# CONVERGENCIA MONTE CARLO
# =====
def analizar_convergencia(simulador):
    """Estudia empíricamente el orden  $O(N^{-1/2})$ ."""
    Ns = np.array([100, 500, 1000, 5000, 10000, 50000])
    media_teo = simulador.esperanza_teorica()

    errores = []
    for N in Ns:
        ST = simulador.precio_terminal_directo(N=N, seed=SEED +
            int(N))
        error = abs(np.mean(ST) - media_teo)
        errores.append(error)

    errores = np.asarray(errores)
    pendiente, intercepto = np.polyfit(np.log(Ns), np.log(
        errores), 1)

    tabla = pd.DataFrame({"N": Ns, "error_absoluto": errores})
    tabla["pendiente_estimacion"] = pendiente
    tabla.to_csv(RESULTADOS / "convergencia_monte_carlo.csv",
        index=False)

    plt.figure(figsize=(7, 5))
    plt.loglog(Ns, errores, "o-", label="Error empírico")
    referencia = np.exp(intercepto) * Ns ** (-0.5)
    plt.loglog(Ns, referencia, "--", label=r"Referencia  $N^{-1/2}$ ")

```

```

plt.xlabel("N mero de simulaciones N")
plt.ylabel("Error absoluto de la media")
plt.title(f"Convergencia Monte Carlo. Pendiente = {pendiente
:.3f}")

plt.legend()
plt.tight_layout()
plt.savefig(FIGURAS / "convergencia_monte_carlo.png", dpi
=200)
plt.close()

return pendiente, tabla

# =====
# AN LISIS DE SENSIBILIDAD
# =====
def sensibilidad(S0, T=1.0, pasos=252):
    """
    Analiza expl citamente media y varianza terminal frente a
    mu y sigma.
    """
    mus = [0.02, 0.05, 0.08, 0.12]
    sigmas = [0.10, 0.18, 0.25, 0.35]
    sigma_fija = 0.18
    mu_fijo = 0.08

    filas_mu = []
    for mu in mus:
        sim = SimuladorGBM(S0=S0, mu=mu, sigma=sigma_fija,
                           T=T, pasos=pasos)
        _, S = sim.simular(N=N_SIMULACIONES, seed=SEED)
        ST = S[:, -1]
        filas_mu.append(
            {
                "mu": mu,
                "media_simulada": np.mean(ST),
                "media_teorica": sim.esperanza_teorica(),
                "varianza_simulada": np.var(ST, ddof=1),
                "varianza_teorica": sim.varianza_teorica(),
                "p5": np.percentile(ST, 5),
                "p95": np.percentile(ST, 95),
            }
        )

```

```

        }
    )

filas_sigma = []
for sigma in sigmas:
    sim = SimuladorGBM(S0=S0, mu=mu_fijo, sigma=sigma,
                       T=T, pasos=pasos)
    _, S = sim.simular(N=N_SIMULACIONES, seed=SEED)
    ST = S[:, -1]
    filas_sigma.append(
        {
            "sigma": sigma,
            "media_simulada": np.mean(ST),
            "media_teorica": sim.esperanza_teorica(),
            "varianza_simulada": np.var(ST, ddof=1),
            "varianza_teorica": sim.varianza_teorica(),
            "p5": np.percentile(ST, 5),
            "p95": np.percentile(ST, 95),
        }
    )

df_mu = pd.DataFrame(filas_mu)
df_sigma = pd.DataFrame(filas_sigma)

df_mu.to_csv(RESULTADOS / "sensibilidad_mu.csv", index=False)
df_sigma.to_csv(RESULTADOS / "sensibilidad_sigma.csv", index=False)

# Varianza frente a mu
plt.figure(figsize=(7, 5))
plt.plot(df_mu["mu"], df_mu["varianza_simulada"], "o-",
         label="Simulada")
plt.plot(df_mu["mu"], df_mu["varianza_teorica"], "s--",
         label="Teórica")
plt.xlabel(r"$\mu$")
plt.ylabel(r"Varianza de $S_T$")
plt.title(r"Sensibilidad de la varianza respecto a $\mu$")
plt.legend()
plt.tight_layout()

```

```

plt.savefig(FIGURAS / "sensibilidad_mu_varianza.png", dpi
            =200)
plt.close()

# Varianza frente a sigma
plt.figure(figsize=(7, 5))
plt.plot(df_sigma["sigma"], df_sigma["varianza_simulada"], "
         o-",
         label="Simulada")
plt.plot(df_sigma["sigma"], df_sigma["varianza_teorica"], "s
         --",
         label="Teórica")
plt.xlabel(r"$\sigma$")
plt.ylabel(r"Varianza de $S_T$")
plt.title(r"Sensibilidad de la varianza respecto a $\sigma$")
plt.legend()
plt.tight_layout()
plt.savefig(FIGURAS / "sensibilidad_sigma_varianza.png", dpi
            =200)
plt.close()

return df_mu, df_sigma

def main():
    (
        df,
        observaciones_originales,
        faltantes_originales,
        duplicados_originales,
    ) = cargar_datos()

    info = descripcion_datos(df)

    graficas_descriptivas(df)

    tabla_normalidad = pruebas_normalidad(df)

    print("\n=== CALIDAD DE LOS DATOS ORIGINALES ===")

```

```

print(f"Observaciones originales: {observaciones_originales}
    ")
print(f"Precios faltantes: {faltantes_originales}")
print(f"Fechas duplicadas: {duplicados_originales}")

for k, v in info.items():
    print(f"{k}: {v}")

print("\n=== PRUEBAS DE NORMALIDAD ===")

print(
    tabla_normalidad.to_string(
        index=False
    )
)

train, test = dividir_train_test(df)

cal = calibrar_gbm(
    train["Price"].to_numpy(),
    sesiones_anuales=SESIONES_ANUALES
)

S0 = float(
    train["Price"].iloc[-1]
)

horizonte = (
    len(test)
    / SESIONES_ANUALES
)

simulador = SimuladorGBM(
    S0=S0,
    mu=cal["mu"],
    sigma=cal["sigma"],
    T=horizonte,
    pasos=len(test),
)

graficar_trayectorias(simulador)

```

```

print("\n=== CALIBRACION GBM ===")

print(
    f"mu = {cal['mu']:.6f}"
)

print(
    f"sigma = {cal['sigma']:.6f}"
)

print("\n=== VALIDACION TEORICA ===")

print(
    validar_distribucion(
        simulador
    ).to_string(index=False)
)

pendiente, _ = analizar_convergencia(
    simulador
)

print(
    f"\nPendiente de convergencia estimada: "
    f"{pendiente:.4f}"
)

sensibilidad(
    S0=float(
        df["Price"].iloc[0]
    )
)

t = (
    np.arange(
        1,
        len(test) + 1
    )
    / SESIONES_ANUALES
)

```

```

pred_media = (
    S0
    * np.exp(
        cal["mu"] * t
    )
)

met = metricas(
    test["Price"].to_numpy(),
    pred_media
)

pd.DataFrame([met]).to_csv(
    RESULTADOS / "metricas_gbm_validacion.csv",
    index=False
)

print("\n=== M TRICAS GBM FUERA DE MUESTRA ===")

for k, v in met.items():
    print(f"{k}: {v:.6f}")

if __name__ == "__main__":
    main()

```

A.3. Implementación del modelo GARCH(1,1)

Listing 3: Código fuente del fichero modelo_garch.py

```

"""
modelo_garch.py
=====
Ajuste, predicci n y simulaci n de un modelo GARCH(1,1).

Dependencias:
    pip install arch pandas numpy matplotlib
"""

from pathlib import Path

```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from arch import arch_model

BASE = Path(__file__).resolve().parent
DATA_PATH = BASE.parent / "datos" / "datos_sp500.csv"
RESULTADOS = BASE.parent / "resultados"
FIGURAS = BASE.parent / "figuras"
RESULTADOS.mkdir(exist_ok=True)
FIGURAS.mkdir(exist_ok=True)

SESIONES_ANUALES = 252
SEED = 42

def cargar_datos(path=DATA_PATH):
    df = pd.read_csv(path)
    df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

    precio = (
        df["Price"]
        .astype(str)
        .str.replace(",", "", regex=False)
        .str.replace(" ", "", regex=False)
    )
    df["Price"] = pd.to_numeric(precio, errors="coerce")

    return (
        df.dropna(subset=["Date", "Price"])
        .sort_values("Date")
        .drop_duplicates(subset="Date")
        .reset_index(drop=True)
    )

def rendimientos_logaritmicos(precios):
    return np.log(precios).diff().dropna()

```

```

def ajustar_garch(rendimientos, distribucion="normal"):
    """
    Ajusta un GARCH(1,1) con media constante.

    Se multiplican los rendimientos por 100 para trabajar en
    porcentaje,
    práctica habitual al utilizar la librería arch.
    """
    r_pct = 100.0 * np.asarray(rendimientos, dtype=float)

    modelo = arch_model(
        r_pct,
        mean="Constant",
        vol="GARCH",
        p=1,
        q=1,
        dist=distribucion,
        rescale=False,
    )
    resultado = modelo.fit(dispatch="off")
    return resultado

def extraer_parametros(resultado):
    p = resultado.params
    return {
        "mu_diario": float(p.get("mu", 0.0) / 100.0),
        "omega": float(p["omega"] / 10000.0),
        "alpha": float(p["alpha[1]"]),
        "beta": float(p["beta[1]"]),
        "alpha_mas_beta": float(p["alpha[1]"] + p["beta[1]"]),
    }

def pronostico_garch(resultado, horizonte):
    """
    Pronóstico condicional de media y volatilidad para '
    horizonte' días.

    Devuelve la media y la desviación típica de los
    rendimientos en escala
    """

```

```

decimal, no en porcentaje.
"""
forecast = resultado.forecast(horizon=horizonte, reindex=
    False)

media = forecast.mean.values[-1] / 100.0
varianza = forecast.variance.values[-1] / 10000.0
sigma = np.sqrt(varianza)

return media, sigma

def simular_garch_precio(
    resultado,
    S0,
    horizonte,
    n_simulaciones=10000,
    seed=SEED,
):
    """
    Genera trayectorias de rendimientos y precios usando la
    dinámica GARCH(1,1).

    La simulación se realiza en escala decimal:
        
$$r_t = \mu + \epsilon_t$$

        
$$\epsilon_t = \sigma_t z_t$$

        
$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$


    Para iniciar la simulación fuera de muestra se calcula
    previamente la
    varianza condicional del primer periodo futuro utilizando
    el último
    residuo observado y la última varianza condicional estimada
    .
    """
    par = extraer_parametros(resultado)
    rng = np.random.default_rng(seed)

    mu = par["mu_diario"]
    omega = par["omega"]

```

```

alpha = par["alpha"]
beta = par["beta"]

#
=====

# CONDICION INICIAL PARA LA SIMULACION FUERA DE MUESTRA
#
=====

# ltima volatilidad condicional estimada (escala decimal)
sigma_T = float(np.asarray(resultado.conditional_volatility)
    [-1]) / 100.0
sigma2_T = sigma_T ** 2

# ltimo residuo observado (escala decimal)
eps_T = float(np.asarray(resultado.resid)[-1]) / 100.0

# Varianza condicional del primer periodo futuro:
#  $\sigma_{T+1}^2 = \omega + \alpha * \epsilon_T^2 + \beta * \sigma_T^2$ 
sigma2_0 = (
    omega
    + alpha * eps_T ** 2
    + beta * sigma2_T
)

#
=====

# SIMULACION
#
=====

precios = np.empty((n_simulaciones, horizonte + 1))
precios[:, 0] = S0

# La primera simulacion utiliza  $\sigma_{T+1}^2$ 
sigma2 = np.full(n_simulaciones, sigma2_0)

```

```

for t in range(1, horizonte + 1):
    z = rng.standard_normal(n_simulaciones)

    sigma = np.sqrt(np.maximum(sigma2, 1e-16))
    eps = sigma * z
    r = mu + eps

    # Actualizaci n del precio mediante rendimientos
    # logar tmicos
    precios[:, t] = precios[:, t - 1] * np.exp(r)

    # Actualizaci n de la varianza para el siguiente
    # per odo
    sigma2 = (
        omega
        + alpha * eps ** 2
        + beta * sigma2
    )

return precios

def main():
    df = cargar_datos()
    n_train = int(len(df) * 0.80)
    train = df.iloc[:n_train].copy()
    test = df.iloc[n_train:].copy()

    r_train = rendimientos_logaritmicos(train["Price"])

    resultado = ajustar_garch(r_train)
    par = extraer_parametros(resultado)

    print(resultado.summary())
    print("\n=== PAR METROS GARCH(1,1) ===")
    for k, v in par.items():
        print(f"{k}: {v:.8f}")

    pd.DataFrame([par]).to_csv(
        RESULTADOS / "parametros_garch.csv", index=False
    )

```

```

# Volatilidad condicional hist rica
fechas_r = train["Date"].iloc[1:].to_numpy()
vol_anual = (
    np.asarray(resultado.conditional_volatility)
    / 100.0
    * np.sqrt(SESIONES_ANUALES)
)

plt.figure(figsize=(10, 5))
plt.plot(fechas_r, vol_anual)
plt.xlabel("Fecha")
plt.ylabel("Volatilidad anualizada")
plt.title("Volatilidad condicional estimada por GARCH(1,1)")
plt.tight_layout()
plt.savefig(FIGURAS / "volatilidad_garch.png", dpi=200)
plt.close()

media_f, sigma_f = pronostico_garch(resultado, len(test))
tabla_forecast = pd.DataFrame(
    {
        "paso": np.arange(1, len(test) + 1),
        "media_rendimiento": media_f,
        "sigma_condicional": sigma_f,
    }
)

tabla_forecast.to_csv(
    RESULTADOS / "pronostico_garch.csv", index=False
)

S0 = float(train["Price"].iloc[-1])
precios_sim = simular_garch_precio(
    resultado,
    S0=S0,
    horizonte=len(test),
    n_simulaciones=10000,
    seed=SEED,
)

np.save(RESULTADOS / "trayectorias_garch.npy", precios_sim)
print("\nTrayectorias GARCH guardadas correctamente.")

```

```
if __name__ == "__main__":  
    main()
```

A.4. Comparación entre modelos

Listing 4: Código fuente del fichero `comparacion_modelos.py`

```
"""  
comparacion_modelos.py  
=====
```

Comparaci n GBM vs GARCH(1,1) usando:

- el mismo S&P 500;
- el mismo conjunto de entrenamiento;
- el mismo conjunto de validaci n;
- el mismo precio inicial;
- el mismo horizonte temporal.

Para instalar dependencias:

```
    pip install numpy pandas matplotlib arch scipy
```

```
"""
```

```
from pathlib import Path  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from scipy.stats import norm
```

```
from simulador_gbm import SimuladorGBM, calibrar_gbm  
from modelo_garch import (  
    cargar_datos,  
    rendimientos_logaritmicos,  
    ajustar_garch,  
    extraer_parametros,  
    simular_garch_precio,  
)
```

```
BASE = Path(__file__).resolve().parent  
RESULTADOS = BASE.parent / "resultados"  
FIGURAS = BASE.parent / "figuras"
```

```

RESULTADOS.mkdir(exist_ok=True)
FIGURAS.mkdir(exist_ok=True)

SESIONES_ANUALES = 252
SEED = 42
N_SIMULACIONES = 10_000

def metricas(y_real, y_pred):
    y_real = np.asarray(y_real, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)

    rmse = np.sqrt(np.mean((y_real - y_pred) ** 2))
    mae = np.mean(np.abs(y_real - y_pred))
    mape = 100.0 * np.mean(
        np.abs((y_real - y_pred) / y_real)
    )
    return {"RMSE": rmse, "MAE": mae, "MAPE": mape}

def cobertura(y_real, inferior, superior):
    y_real = np.asarray(y_real)
    inferior = np.asarray(inferior)
    superior = np.asarray(superior)
    return 100.0 * np.mean(
        (y_real >= inferior) & (y_real <= superior)
    )

def precio_black_scholes_call(S0, K, r, sigma, T):
    """Precio analítico de una call europea Black--Scholes."""
    d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma *
        np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S0 * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def comparacion_black_scholes(S0, sigma, seed=SEED):

```

```

"""Compara valoraci n Monte Carlo bajo GBM con Black--
    Scholes."""
K, r, T = S0, 0.04, 1.0
Ns = np.array([100, 500, 1000, 5000, 10000, 50000])
precio_bs = precio_black_scholes_call(S0, K, r, sigma, T)
estimaciones = []
rng = np.random.default_rng(seed)
ST_final = None
for N in Ns:
    Z = rng.standard_normal(N)
    ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.
        sqrt(T) * Z)
    precio_mc = np.exp(-r * T) * np.mean(np.maximum(ST - K,
        0.0))
    estimaciones.append(precio_mc)
    if N == Ns[-1]: ST_final = ST
estimaciones = np.asarray(estimaciones)
tabla = pd.DataFrame({'N':Ns,'Monte_Carlo':estimaciones,'
    Black_Scholes':precio_bs,'Error_absoluto':np.abs(
        estimaciones-precio_bs)})
tabla.to_csv(RESPULTADOS/'
    comparacion_montecarlo_black_scholes.csv',index=False)

plt.figure(figsize=(8,5)); plt.hist(ST_final,bins=60,density
    =True,alpha=0.7)
plt.axvline(S0,linestyle='--',label='Precio inicial'); plt.
    axvline(K,linestyle=':',label='Strike')
plt.xlabel('Precio terminal $S_T$'); plt.ylabel('Densidad');
    plt.title('Distribuci n simulada del precio terminal');
    plt.legend(); plt.tight_layout()
plt.savefig(FIGURAS/'distribucion_precio_terminal.png',dpi
    =220); plt.close()

plt.figure(figsize=(8,5)); plt.semilogx(Ns,estimaciones,'o-'
    ,label='Monte Carlo')
plt.axhline(precio_bs,linestyle='--',label='Black--Scholes')
    ; plt.xlabel('N mero de simulaciones N'); plt.ylabel('
    Precio de la opci n'); plt.title('Convergencia Monte
    Carlo hacia Black--Scholes'); plt.legend(); plt.
    tight_layout()

```

```

plt.savefig(FIGURAS/'convergencia_black_scholes.png',dpi
            =220); plt.close()
return tabla

def main():
    # =====
    # 1. DATOS Y DIVISI N TEMPORAL
    # =====
    df = cargar_datos()
    n_train = int(len(df) * 0.80)

    train = df.iloc[:n_train].copy()
    test = df.iloc[n_train:].copy()

    precios_train = train["Price"].to_numpy()
    precios_test = test["Price"].to_numpy()
    fechas_test = test["Date"].to_numpy()

    S0 = float(precios_train[-1])
    horizonte = len(test)
    T = horizonte / SESIONES_ANUALES

    # =====
    # 2. GBM
    # =====
    cal_gbm = calibrar_gbm(precios_train, SESIONES_ANUALES)

    gbm = SimuladorGBM(
        S0=S0,
        mu=cal_gbm["mu"],
        sigma=cal_gbm["sigma"],
        T=T,
        pasos=horizonte,
    )

    _, S_gbm = gbm.simular(
        N=N_SIMULACIONES,
        seed=SEED,
    )

    gbm_media = S_gbm[:, 1:].mean(axis=0)

```

```

gbm_p5 = np.percentile(S_gbm[:, 1:], 5, axis=0)
gbm_p95 = np.percentile(S_gbm[:, 1:], 95, axis=0)

# =====
# 3. GARCH(1,1)
# =====
r_train = rendimientos_logaritmicos(train["Price"])
resultado_garch = ajustar_garch(r_train)
par_garch = extraer_parametros(resultado_garch)

S_garch = simular_garch_precio(
    resultado_garch,
    S0=S0,
    horizonte=horizonte,
    n_simulaciones=N_SIMULACIONES,
    seed=SEED,
)

garch_media = S_garch[:, 1:].mean(axis=0)
garch_p5 = np.percentile(S_garch[:, 1:], 5, axis=0)
garch_p95 = np.percentile(S_garch[:, 1:], 95, axis=0)

# =====
# 4. M TRICAS
# =====
met_gbm = metricas(precios_test, gbm_media)
met_garch = metricas(precios_test, garch_media)

cov_gbm = cobertura(precios_test, gbm_p5, gbm_p95)
cov_garch = cobertura(precios_test, garch_p5, garch_p95)

tabla = pd.DataFrame(
    {
        "M trica": ["RMSE", "MAE", "MAPE (%)", "Cobertura
                    5-95 (%)"],
        "GBM": [
            met_gbm["RMSE"],
            met_gbm["MAE"],
            met_gbm["MAPE"],
            cov_gbm,
        ],
    },

```

```

        "GARCH(1,1)": [
            met_garch["RMSE"],
            met_garch["MAE"],
            met_garch["MAPE"],
            cov_garch,
        ],
    }
)

tabla.to_csv(
    RESULTADOS / "comparacion_gbm_garch.csv",
    index=False,
)

print("\n=== COMPARACION GBM VS GARCH(1,1) ===")
print(tabla.to_string(index=False))

print("\n=== PARAMETROS ===")
print(f"GBM mu anual: {cal_gbm['mu']:.6f}")
print(f"GBM sigma anual: {cal_gbm['sigma']:.6f}")
for k, v in par_garch.items():
    print(f"GARCH {k}: {v:.8f}")

# =====
# 5. GR FICO: PRECIO REAL Y MEDIAS SIMULADAS
# =====
plt.figure(figsize=(11, 6))
plt.plot(fechas_test, precios_test, label="Precio real")
plt.plot(fechas_test, gbm_media, label="Media GBM")
plt.plot(fechas_test, garch_media, label="Media GARCH(1,1)")

plt.fill_between(
    fechas_test,
    gbm_p5,
    gbm_p95,
    alpha=0.15,
    label="Banda GBM 5-95 %",
)

plt.fill_between(
    fechas_test,

```

```

        garch_p5,
        garch_p95,
        alpha=0.15,
        label="Banda GARCH 5-95 %",
    )

plt.xlabel("Fecha")
plt.ylabel("Precio")
plt.title("Comparaci n fuera de muestra: GBM vs GARCH(1,1)"
    )
plt.legend()
plt.tight_layout()
plt.savefig(
    FIGURAS / "comparacion_precios_gbm_garch.png",
    dpi=220,
)
plt.close()

# =====
# 6. GR FICO: VOLATILIDAD
# =====
# Volatilidad condicional estimada en entrenamiento
fechas_vol = train["Date"].iloc[1:].to_numpy()
vol_garch = (
    np.asarray(resultado_garch.conditional_volatility)
    / 100.0
    * np.sqrt(SESIONES_ANUALES)
)

plt.figure(figsize=(11, 5))
plt.plot(
    fechas_vol,
    vol_garch,
    label="Volatilidad condicional GARCH",
)
plt.axhline(
    cal_gbm["sigma"],
    linestyle="--",
    label="Volatilidad constante GBM",
)
plt.xlabel("Fecha")

```

```

plt.ylabel("Volatilidad anualizada")
plt.title("Volatilidad: GARCH(1,1) frente a GBM")
plt.legend()
plt.tight_layout()
plt.savefig(
    FIGURAS / "comparacion_volatilidad_gbm_garch.png",
    dpi=220,
)
plt.close()

# =====
# 7. GUARDAR SERIES PARA LATEX O ANALISIS POSTERIOR
# =====
series = pd.DataFrame(
    {
        "Date": fechas_test,
        "Precio_real": precios_test,
        "GBM_media": gbm_media,
        "GBM_p5": gbm_p5,
        "GBM_p95": gbm_p95,
        "GARCH_media": garch_media,
        "GARCH_p5": garch_p5,
        "GARCH_p95": garch_p95,
    }
)

series.to_csv(
    RESULTADOS / "series_comparacion_gbm_garch.csv",
    index=False,
)

# 8. COMPARACION MONTE CARLO (GBM) VS BLACK--SCHOLES
tabla_bs = comparacion_black_scholes(S0, cal_gbm["sigma"])
print("\n=== MONTE CARLO (GBM) VS BLACK--SCHOLES ===")
print(tabla_bs.to_string(index=False))

print("\nArchivos generados en:")
print(RESULTADOS)
print(FIGURAS)

```

```
if __name__ == "__main__":  
    main()
```